PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 8

Module 23: Automation

Module 24: Machine Learning Basics

Continues from Parts 1-7 (Modules 1-22)

TABLE OF CONTENTS

MODULE 23: AUTOMATION

One of Python's most practical everyday uses is automating repetitive tasks — things you'd otherwise do by hand, over and over. This module covers three common automation categories: sending emails, working with files in bulk, and scheduling tasks.

23.1 Email Automation

Python's built-in smtplib module can send emails directly from a script — useful for automated reports, alerts, or notifications.

```
import smtplib
from email.mime.text import MIMEText

sender = "you@gmail.com"
receiver = "friend@gmail.com"
password = "your_app_password"   # use an app password, never your real password

msg = MIMEText("This is an automated email sent from Python.")
msg["Subject"] = "Automated Report"
msg["From"] = sender
msg["To"] = receiver

with smtplib.SMTP("smtp.gmail.com", 587) as server:
    server.starttls()             # secure the connection
    server.login(sender, password)
    server.send_message(msg)
    print("Email sent successfully!")
```

Security Note

Never hardcode your real email password in a script. Modern email providers (like Gmail) require a separate 'app password' generated specifically for scripts, and even that should be stored in an environment variable, not directly in the code.

23.2 File Automation

The os and shutil modules let Python rename, move, copy, and organise files automatically — useful for cleaning up a messy downloads folder, for example.

```
import os
import shutil

# Rename all .txt files in a folder by adding a prefix
folder = "documents"
for filename in os.listdir(folder):
    if filename.endswith(".txt"):
        old_path = os.path.join(folder, filename)
        new_path = os.path.join(folder, "processed_" + filename)
        os.rename(old_path, new_path)

# Move all .jpg files into an "images" subfolder
os.makedirs("documents/images", exist_ok=True)
for filename in os.listdir(folder):
    if filename.endswith(".jpg"):
        shutil.move(os.path.join(folder, filename), "documents/images")
```

23.3 Task Automation

The schedule library (or the operating system's own scheduler — Task Scheduler on Windows, cron on Linux/macOS) can run a Python script automatically at set times, without you manually starting it.

```
# pip install schedule
import schedule
import time

def daily_task():
    print("Running the daily backup task...")

schedule.every().day.at("09:00").do(daily_task)

while True:
    schedule.run_pending()
    time.sleep(60)   # check once per minute
```

Real-World Analogy

Task automation is like setting a daily alarm clock — instead of you remembering to run a script every morning, the scheduler 'rings' at the right time and runs it for you automatically.

Real Project — Bulk File Organiser

```
import os, shutil

FOLDER = "downloads"
FILE_TYPES = {
    "images": [".jpg", ".png",
".gif"],
    "documents": [".pdf", ".docx",
".txt"],
    "videos": [".mp4", ".mov"],
}

for filename in
os.listdir(FOLDER):
    filepath =
os.path.join(FOLDER, filename)
    if os.path.isfile(filepath):
        for category, extensions
in FILE_TYPES.items():
            if
any(filename.lower().endswith(ext)
for ext in extensions):
                dest =
os.path.join(FOLDER, category)
                os.makedirs(dest,
exist_ok=True)

shutil.move(filepath, dest)
                break
```

Line	Explanation
4-8	Define a mapping of category name to the file extensions that belong to it.
10-11	Loop through every item in the downloads folder, checking it's actually a file (not a subfolder).
13-14	Check each category's extensions to see if the current filename matches one.
15-17	Create the destination category folder if needed, and move the matching file into it.

Interview Questions — Module 23

Q1. Why should you use an 'app password' instead of your real password for email automation?

Ans: App passwords are limited-scope credentials generated specifically for a script/application, so if they are ever exposed, the damage is contained — your main account password and its full access remain safe.

Q2. Which modules are commonly used for file automation in Python?

Ans: os (for path handling, renaming, listing directories) and shutil (for higher-level operations like copying and moving files/folders).

Q3. What is the purpose of exist_ok=True in os.makedirs()?

Ans: It prevents an error from being raised if the target folder already exists — without it, calling makedirs() on an existing path raises a FileExistsError.

Q4. What is the benefit of automating repetitive tasks with Python?

Ans: It saves time, reduces the risk of human error in repetitive steps, and ensures tasks run consistently and on schedule without requiring someone to remember to do them manually.

Q5. How can a Python script be made to run automatically every day?

Ans: Using a scheduling library like schedule within a long-running script, or (more robustly, for production use) registering the script with the operating system's own scheduler — Task Scheduler on Windows or cron on Linux/macOS.

MCQs — Module 23

No.	Question	Answer	Options (A-D)
1	Which module sends emails from Python?	**C**	A) email B) mailer C) smtplib D) sendmail
2	Which module is used for higher-level file operations like moving files?	**B**	A) os B) shutil C) sys D) email
3	What should you NEVER hardcode directly in an automation script?	**D**	A) File paths B) Folder names C) Function names D) Passwords/API keys
4	Which library helps schedule a Python function to run daily?	**A**	A) schedule B) time C) calendar D) cron (Python module)
5	exist_ok=True in os.makedirs() prevents an error when:	**C**	A) The disk is full B) Permission denied C) The folder already exists D) The path is invalid

Exercises — Module 23

1. Write a script to rename all files in a folder by adding today's date as a prefix.
2. Write a script to automatically sort files in a folder into subfolders by file extension.
3. Write a script using the schedule library to print a reminder message every hour.
4. Write a script to find and delete all empty files in a given folder.

MODULE 24: MACHINE LEARNING BASICS

Machine Learning (ML) is a branch of AI where a program learns patterns from data instead of being explicitly programmed with fixed rules. This module introduces the core ideas and shows a first working example using scikit-learn, Python's most popular ML library.

> **Setup**
> Install once with: pip install scikit-learn Then import with: from sklearn import ...

24.1 Introduction

Traditional programming: you write explicit rules, and the program produces output from input following those rules. Machine learning flips this: you give the program many examples of input AND correct output, and it learns the underlying rules (patterns) itself.

```
Traditional Programming:      Input + Rules  --------> Output
Machine Learning:              Input + Output --------> Rules (the model)
```

Broad categories of Machine Learning:

Type	Meaning	Example	Common Task
Supervised Learning	Learns from labelled data (input + correct answer)	Predicting house prices from features	Regression, Classification
Unsupervised Learning	Finds patterns in unlabelled data	Grouping customers by behaviour	Clustering
Reinforcement Learning	Learns via trial, error, and rewards	A game-playing agent	Game AI, robotics

24.2 Data Preprocessing

Real-world data is rarely ready to use directly — it often needs cleaning and preparation before a model can learn from it effectively.

- Handling missing values: filling them in (e.g., with the column's mean) or removing incomplete rows.
- Encoding categorical data: converting text categories (like 'Yes'/'No') into numbers, since ML algorithms need numeric input.
- Feature scaling: bringing all numeric columns to a similar range, so no single feature unfairly dominates just because its numbers are naturally larger.
- Splitting data: dividing the dataset into a training set (to teach the model) and a testing set (to fairly evaluate it on unseen data).

```
from sklearn.model_selection import train_test_split

X = [[1],[2],[3],[4],[5],[6],[7],[8]]   # feature: hours studied
y = [35, 40, 50, 55, 65, 70, 80, 90]      # target: marks scored

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=1)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Output:

```
Training samples: 6
Testing samples: 2
```

24.3 Linear Regression

Linear regression predicts a continuous numeric value (e.g., price, marks, temperature) by fitting a straight line through the data that best captures the relationship between input(s) and output.

```
from sklearn.linear_model import LinearRegression

X = [[1],[2],[3],[4],[5]]   # hours studied
y = [35, 45, 55, 65, 75]     # marks scored

model = LinearRegression()
model.fit(X, y)               # train the model

prediction = model.predict([[6]])   # predict marks for 6 hours studied
print("Predicted marks:", prediction[0])
```

Output:

```
Predicted marks: 85.0
```

Visual intuition (illustrative sketch):

```
Marks
90 |                          *  <- predicted
75 |                     *
65 |                *
55 |          *
45 |   *
35 |*
   +---------------------------
     1   2   3   4   5   6  Hours
  (best-fit line through the actual data points)
```

24.4 Classification

Classification predicts a category/label (e.g., spam vs not-spam, pass vs fail) rather than a continuous number. A simple, popular algorithm for this is K-Nearest Neighbours (KNN).

```
from sklearn.neighbors import KNeighborsClassifier

# Features: [hours_studied, attendance_%]
X = [[2, 60], [8, 90], [1, 50], [9, 95], [3, 65], [7, 85]]
# Labels: 0 = Fail, 1 = Pass
y = [0, 1, 0, 1, 0, 1]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

result = model.predict([[6, 80]])   # new student: 6 hrs studied, 80% attendance
print("Pass" if result[0] == 1 else "Fail")
```

Output:

```
Pass
```

Common Beginner Mistake

Evaluating a model only on the same data it was trained on gives a falsely optimistic result — the model may have simply memorised the training examples. Always test on a separate, unseen test set (using train_test_split) to get a fair measure of real performance.

Interview Questions — Module 24

Q1. What is the difference between supervised and unsupervised learning?

Ans: Supervised learning trains on labelled data (each example has a known correct answer), used for tasks like regression and classification. Unsupervised learning works on unlabelled data and tries to find hidden patterns or groupings on its own, such as clustering.

Q2. Why do we split data into training and testing sets?

Ans: Training data teaches the model, while a separate, unseen testing set lets us fairly evaluate how well the model generalises to new data, rather than just checking if it memorised the training examples.

Q3. What is the difference between regression and classification?

Ans: Regression predicts a continuous numeric value (like a price or a score). Classification predicts a discrete category or label (like spam/not-spam, or pass/fail).

Q4. What is feature scaling and why is it needed?

Ans: It brings numeric features onto a comparable range (e.g., 0 to 1). Without it, features with naturally larger numeric ranges can dominate a model's calculations simply due to their scale, not their actual importance.

Q5. What does model.fit() do in scikit-learn?

Ans: It trains the model — the algorithm analyses the given input (X) and output (y) training data and learns the underlying pattern/relationship, which is then stored inside the model object for future predictions.

Q6. What is overfitting?

Ans: When a model learns the training data too closely — including its noise and specific quirks — so it performs very well on training data but poorly on new, unseen data, because it failed to learn the general underlying pattern.

MCQs — Module 24

No.	Question	Answer	Options (A-D)
1	Which library is commonly used for basic ML in Python?	**B**	A) matplotlib B) scikit-learn C) requests D) sqlite3
2	Predicting a numeric value like house price is an example of:	**C**	A) Classification B) Clustering C) Regression D) Scraping
3	Predicting Pass/Fail is an example of:	**A**	A) Classification B) Regression C) Clustering D) Scraping
4	Which function splits data into training and testing sets?	**D**	A) split_data() B) train_model() C) data_split() D) train_test_split()
5	Which method trains a scikit-learn model?	**C**	A) train() B) learn() C) fit() D) build()
6	Evaluating a model only on its training data risks:	**B**	A) Underfitting B) A falsely optimistic result C) Faster training D) Lower accuracy always

Exercises — Module 24

1. Use LinearRegression to predict exam marks based on hours studied, using your own small dataset.
2. Use train_test_split to split a dataset of 20 samples into 80% training and 20% testing.
3. Use KNeighborsClassifier to classify whether a fruit is an apple or orange based on weight and colour score.
4. Explain, in your own words, the difference between training data and testing data.

QUICK REVISION NOTES — MODULES 23-24

- smtplib sends emails (use app passwords, never real ones); os/shutil automate file organisation; schedule automates recurring tasks.
- Never hardcode passwords/API keys in automation scripts — use environment variables.
- ML flips traditional programming: instead of writing rules, the model learns rules from labelled examples (input + correct output).
- Supervised learning = labelled data (regression for numbers, classification for categories). Unsupervised = unlabelled data (clustering).
- Always split data into training and testing sets (train_test_split) to fairly evaluate a model's real-world performance.
- LinearRegression predicts continuous values; KNeighborsClassifier (and similar) predicts categories.
- model.fit(X, y) trains a scikit-learn model; model.predict(new_X) generates predictions on new data.

COMMON ERRORS AND FIXES — MODULES 23-24

Error	Cause	Fix
smtplib.SMTPAuthenticationError	Using your real password instead of an app password	Generate and use an app-specific password
FileExistsError	Creating a folder that already exists	Use os.makedirs(path, exist_ok=True)
ModuleNotFoundError: sklearn	scikit-learn not installed	Run: pip install scikit-learn
ValueError: Found input variables with inconsistent numbers of samples	X and y lists have different lengths	Ensure every input row has a matching label
Model performs poorly on new data	Overfitting, or evaluated only on training data	Always test on a separate, unseen test set

CODING BEST PRACTICES — MODULES 23-24

- Always test automation scripts on sample/dummy data first, especially file-deleting or bulk-renaming scripts, before running on real data.
- Log automation actions (what was renamed/moved/sent) so you can review and undo mistakes if needed.
- Store credentials in environment variables, never directly in automation scripts.
- Always evaluate ML models on a held-out test set, never just the training data.
- Start with simple models (like linear regression or KNN) before reaching for complex ones — they're easier to understand and debug.